Part A: Understanding NoSQL and MongoDB

1. What is NoSQL?

NoSQL (Not Only SQL) is a type of database that stores data in a **non-tabular format** such as documents, key-value pairs, graphs, or wide-column stores. It is designed for **handling large volumes of unstructured or semi-structured data** and provides **high scalability and flexibility**.

---

2. How is NoSQL different from SQL databases?

SQL Database	NoSQL Database
Uses tables with rows and columns	Uses documents, key-value, graph, or column formats
Fixed schema	Dynamic schema
Vertical scaling	Horizontal scaling
Examples: MySQL, PostgreSQL	Examples: MongoDB, Cassandra

---

3. What is MongoDB?

MongoDB is a **NoSQL document-oriented database** that stores data in **JSON-like documents** with flexible schema, making it easy to store and manage large amounts of data.

---

4. What is a document in MongoDB?

A **document** is a single record in MongoDB stored in **BSON format** (similar to JSON).
Example:

```
{
  "name": "Rahul",
  "email": "rahul@gmail.com",
  "age": 21
}
```

---

5. What is a collection?

A **collection** is a group of MongoDB documents.
It is similar to a **table in relational databases**.

Example:

students (collection)

---

6. What is BSON?

BSON (Binary JSON) is the **binary representation of JSON documents** used internally by MongoDB for storing data efficiently.

---

7. What are advantages of MongoDB?

- Flexible schema
- High scalability
- High performance
- Easy integration with JavaScript/Node.js

Part B: MongoDB Practice (Observations)

Insert 3 student records example

{ "name": "Amit", "age": 20, "course": "AIML" }
{ "name": "Priya", "age": 21, "course": "AIML" }
{ "name": "Rohan", "age": 22, "course": "AIML" }

View inserted records

db.students.find()

```
test> db.students.find()
[
  {
    _id: ObjectId('69afcbf6b72cf6996c7c2907'),
    name: 'Amit',
    email: 'amit@gmail.com',
    age: 20
  },
  {
    _id: ObjectId('69afcbf6b72cf6996c7c2908'),
    name: 'Priya',
    email: 'priya@gmail.com',
    age: 22
  },
  {
    _id: ObjectId('69afcbf6b72cf6996c7c2909'),
    name: 'Rohan',
    email: 'rohan@gmail.com',
    age: 21
  }
]
test>
```

Delete one record

db.students.deleteOne({name:"Amit"})

```
]
test> db.students.deleteOne({name:"Amit"})
{ acknowledged: true, deletedCount: 1 }
test>
```

Update one record

db.students.updateOne(
  {name:"Priya"},
  {$set:{age:22}}
)

```
test> db.students.updateOne(
|   {name:"Priya"},
|   {$set:{age:22}}
| )
|
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 0,
  upsertedCount: 0
}
test>
```

Part C: Mongoose Familiarization

What happens if connection string is wrong?

- Database connection fails
- Error is shown in terminal

Example error:

MongoNetworkError: failed to connect to server

What is the purpose of mongoose?

Mongoose is an **ODM (Object Data Modeling) library for Node.js** that helps interact with MongoDB by:

- Defining schemas
- Creating models
- Validating data
- Managing database operations easily

Task 4 Observations

Is the data stored in MongoDB?

Yes. When a record is inserted using a Mongoose model, it is **stored in the MongoDB collection**.

Does schema restrict data structure?

Yes. The **schema defines the structure of documents** and helps enforce validation rules.

Part D: REST API Practice

Status Codes

Operation	Status Code
POST (Create user)	201 Created
GET (Retrieve users)	200 OK
PUT (Update user)	200 OK
DELETE (Delete user)	200 OK

JSON Response Example

```
{
  "message": "User created successfully",
  "data": {
    "name": "John",
    "email": "john@gmail.com",
    "age": 23
  }
}
```

Part E: React API Practice

Does UI update after fetching data?

Yes. When data is fetched from the API and stored in **React state**, the component re-renders and updates the UI.

What happens if backend is not running?

- API request fails
- Error appears in browser console
- No data is displayed in UI

Part F: Data Persistence

Does data still exist after restarting server? Why?

Yes. The data still exists because it is **stored in MongoDB database**, not in the Node.js server memory.

Task 8 Observations

What status codes are returned for each operation?

Operation	Status Code
POST	201
GET	200
PUT	200
DELETE	200

What happens if an invalid ID is used?

- MongoDB cannot find the document
- API returns **404 Not Found** or **400 Bad Request**

Does the database reflect changes immediately?

Yes. MongoDB **updates the database instantly** when CRUD operations are performed.

Task 9: Validation

Does MongoDB allow invalid data without schema?

Yes. MongoDB itself allows flexible data structure, so invalid data can be inserted if no schema validation is applied.

How does Mongoose validation improve data quality?

Mongoose validation ensures:

- Required fields must be present
- Data types are correct
- Values meet defined conditions

This prevents invalid or incomplete data from entering the database.

Where are validation errors displayed?

Validation errors are displayed:

- In the **server terminal**
- In the **API response error message**

Example:

```
{
  "error": "Email is required"
}
```

Task 10: Error Handling

What status code is returned for bad requests?

400 – Bad Request

What happens if MongoDB server is stopped?

- API cannot connect to database
- Requests fail
- Server shows **connection error**

How does proper error handling improve API reliability?

- Prevents application crashes
- Provides clear error messages